Exception Handling

02 April 2026 20:24

Exception handling is one of the most powerful mechanism to handle runtime errors so that normal flow of the application can be maintained.

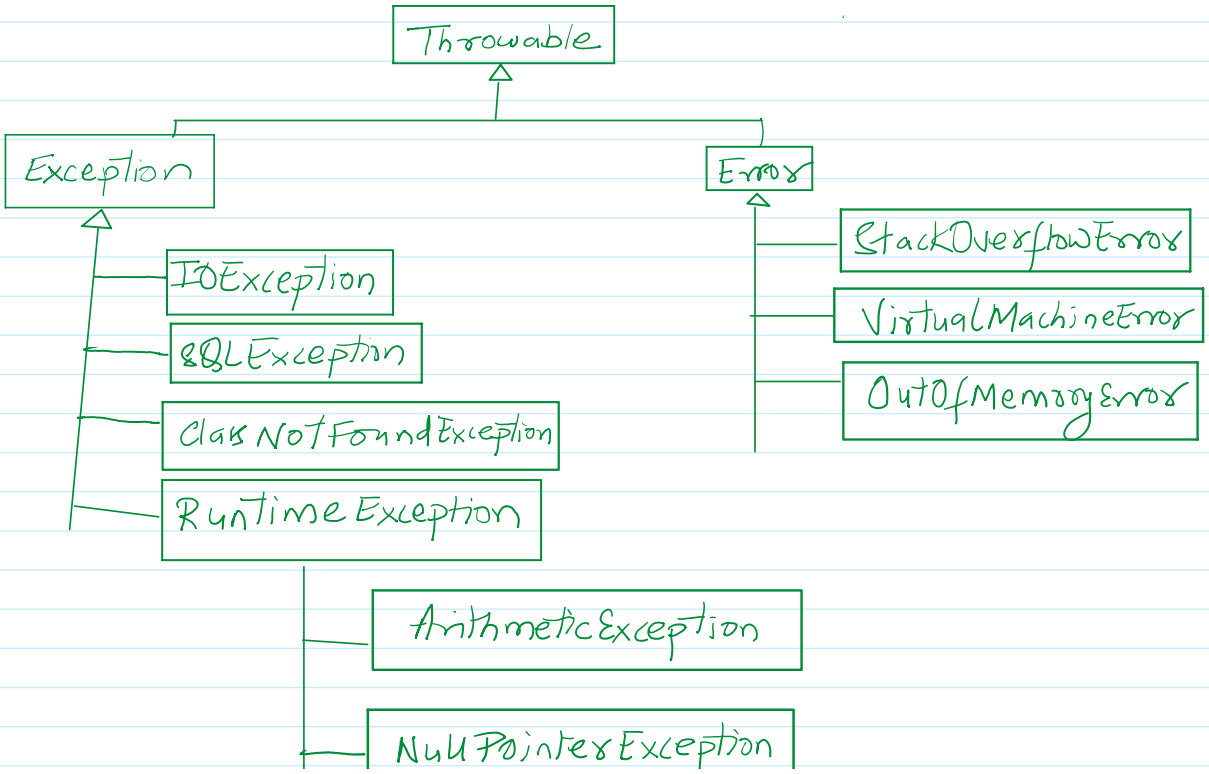
What is Exception in Java?
In Java, an exception is an event that occurs during the execution of a program and disrupt the normal flow of instructions. Exception can occur for various reasons such as invalid user input of file not being found or division by zero error. When an exception occurs, it is typically represented by an object of a subclass of the **java.lang.Exception** class.

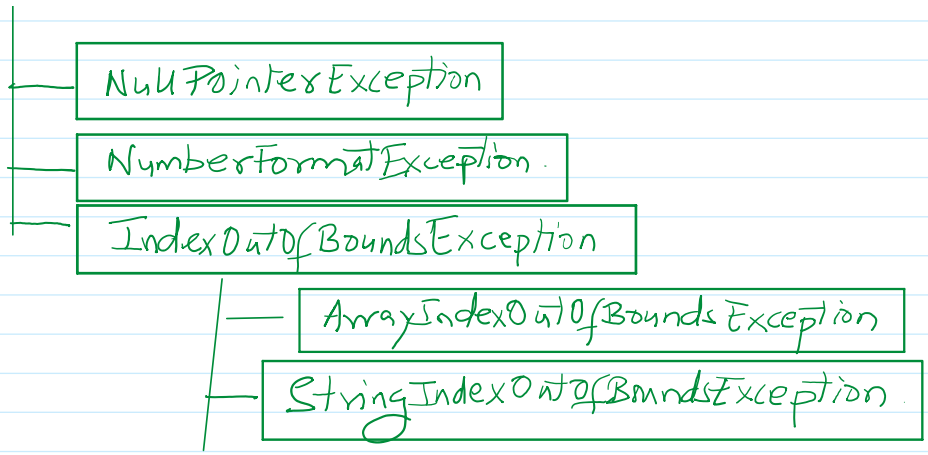
What is Exception Handling in Java?
Exception Handling is a mechanism that allow a program to handle **runtime errors** such as ClassNotFoundException, IOException, SQLException, InputmismatchException, and others, so that normal flow of the application can be maintained.

Hierarchy of Java Exception classes
Exceptions are represented by classes and all exception classes are the part of java.lang. package. They are organized in a hierarchical structure with Throwable at the top. The main hierarchy is as follows:

- **Throwable:** The super class of all errors and exceptions in Java.
  - **Error:** Represents serious system errors that application usually cannot handle (e.g., OutOfMemoryError).
  - **Exception:** Represents conditions that a program might want to catch.
    - **Checked Exception(Compile-time Exception): Must be either handled using try-catch or declared using throws(e.g., IOException, and many more)**

    - **Unchecked Exception (Runtime Exception): Do not need explicit handling, typically caused by programming errors(e.g., NullPointerException, ArithmeticException, and many more)**





Types of Java Exception

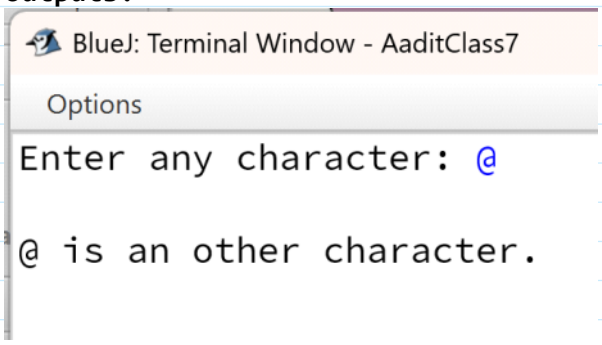
The exceptions are categorized in two main types: checked exceptions and unchecked exceptions. Additionally, there is a 3rd category known as Errors.

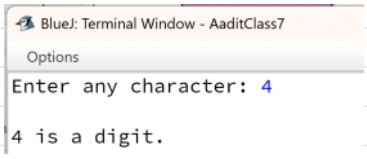
- **Checked Exception:** Exception that must be handled or declared in the code.
- **Unchecked Exception:** Exceptions that do not require explicit handling and usually occur due to programming errors.
- **Error: Serious problems that are not meant to be caught by applications, usually system-related.**

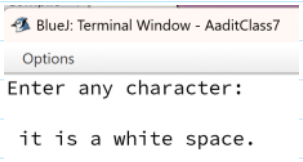
```
package ExceptionHandling;


/**
 * Write a description of class CheckedExceptionExample here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class CheckedExceptionExample
{
    public static void main(String[] args) throws java.io.IOException {
        char c;
        String v="aeiouAEIOU";
        System.out.print("\fEnter any character: ");
        c = (char)System.in.read();
        if(Character.isAlphabetic(c)){
            System.out.print("\n" + c + " is an alphabet");
            if(Character.isUpperCase(c)){
                System.out.print("\n" + c + " is an uppercase alphabet.");
            }
            else{
                System.out.print("\n" + c + " is an lowercase alphabet.");
            }
            if(v.indexOf(c) > -1){
                System.out.print("\n" + c + " is a vowel.");
            }
            else{
                System.out.print("\n" + c + " is not a vowel");
```

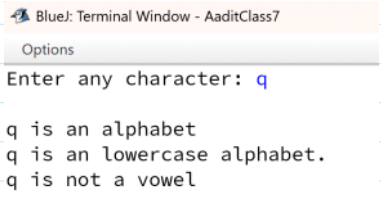
```
            }
        }
        else if(Character.isWhitespace(c)){
            System.out.print("\n it is a white space." );
        }
        else if(Character.isDigit(c)){
            System.out.print("\n" + c + " is a digit.");
        }
        else{
            System.out.print("\n" + c + " is an other character.");
        }

    }
}
```

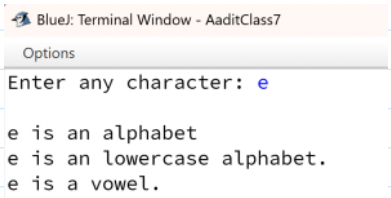
Outputs:











Java unchecked exceptions(runtime exceptions)

Unchecked exceptions, also known as runtime exceptions, are not checked at compile-time. These exceptions usually occur due to programming errors such as logic errors

or incorrect assumption in the code. They do not need to be declared in the method signatures using the throw keyword, making it optional to handle them. Examples of a checked unchecked exceptions include:

- **NullPointerException:** It is thrown when trying to access or call a method on an object reference that is null.

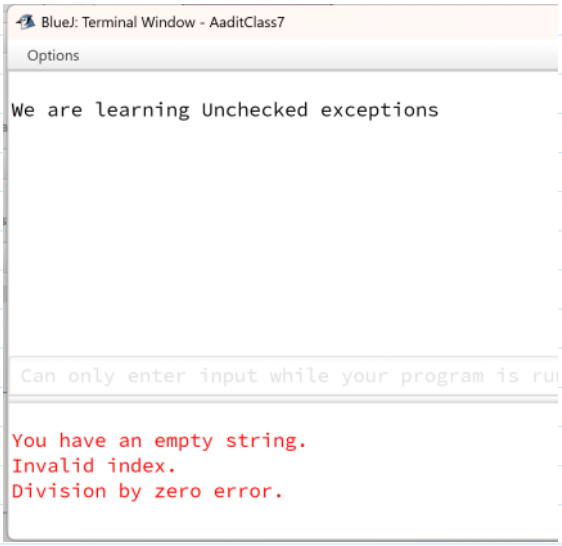
- **ArrayIndexOutOfBoundsException:** It occurs when we try to access an array element with an invalid index.

- **ArithmeticException:** It is thrown when an arithmetic operation fails, such as division by 0.

- **IllegalArgumentException:** It indicates that a method has been passed an illegal or inappropriate argument.

```
package ExceptionHandling;

/**
 * Write a description of class UncheckedExceptionExample here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class UncheckedExceptionExample
{
    public static void main(String[] args){
        String str = null;
        System.out.print("\f");
        try{
            System.out.print("Length of str: "+ + str.length());
        }
        catch(NullPointerException npe){
            System.err.print("\nYou have an empty string.");
        }

        int[] arr = {1,2,3};
        try{
            System.out.print("\n"+arr[5]);
        }
        catch(ArrayIndexOutOfBoundsException aiobe){
            System.err.print("\nInvalid index.");
        }

        try{
            int r = 10/0;
        }
        catch(ArithmeticException ae){
            System.err.print("\nDivision by zero error.");
        }
        System.out.print("\nWe are learning Unchecked exceptions");
    }
}
```

Output:



Java Exception Handling Keywords

Java provides five keywords that are used to handle the exception. The following table describes each.

Keyword	Description
try	The **"try"** keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The **try** block must be followed either catch or finally block.
catch	The **"catch"** block is used to handle the exception. It must be preceded by **"try"** block which means we can't use **"catch"** block alone. It can be followed by **"finally"** block later.
finally	The **"finally"** block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
throw	The throw keyword is used to throw an exception.
throws	The **"throws"** keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It does not throw an exception. It is always used with the method signature.

The try-catch Block

The try - catch block is a primary mechanism for handling exceptions. The tribe lock contains code that may throw an exception, and the catch block handles that exception.

Syntax:

```
    try{
      // Code that may throw an exception.
    }
    catch(ExceptionType e){
      // Code to handle the exception.
    }
   package ExceptionHandling;

   /**
    * Write a description of class UncheckedExceptionExample here.
```

```
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class UncheckedExceptionExample
{
    public static void main(String[] args){
        String str = null;
        System.out.print("\f");
        try{
            System.out.print("Length of str: "+ + str.length());
        }
        catch(NullPointerException npe){
            System.err.print("\nYou have an empty string.");
        }

        int[] arr = {1,2,3};
        try{
            System.out.print("\n"+arr[5]);
        }
        catch(ArrayIndexOutOfBoundsException aiobe){
            System.err.print("\nInvalid index.");
        }

        try{
            int r = 10/0;
        }
        catch(ArithmeticException ae){
            System.err.print("\nDivision by zero error.");
        }
        System.out.print("\nWe are learning Unchecked exceptions");
    }
}
```

Homework for practice:

1. Write a Java program to create a student result management system. The program should accept marks of multiple subjects, calculate the percentage, and determine the grade. Handle exceptions such as invalid marks (less than 0 or greater than 100), division by zero while calculating percentage, and null student names. Use try-catch-finally blocks and create a custom exception called InvalidMarksException.

2. Write a Java program for a bank account system that supports deposit, withdrawal, and balance checking. Throw an exception when the withdrawal amount exceeds the available balance. Also handle invalid amounts such as negative deposits or withdrawals. Use throw, throws, multiple catch blocks, and a custom exception named InsufficientBalanceException.

3. Write a Java program to read data from a text file and display its contents line by line. Handle checked exceptions such as FileNotFoundException and IOException. Ensure that the file stream is closed properly using a finally block.

4. Write a Java program that takes an array of integers from the user and performs operations such as finding the largest number, smallest number, and average. Handle

exceptions such as ArrayIndexOutOfBoundsException, InputMismatchException, and ArithmeticException.

5. Write a Java program to simulate an online shopping cart. The program should allow the user to add products, remove products, and calculate the total price. Handle exceptions for invalid product quantity, negative price, and null product names. Create a custom exception named InvalidProductException.

6. Write a Java program that demonstrates nested try-catch blocks. The program should:
   - Accept two numbers and divide them
   - Store the result in an array
   - Access an array element using a user-provided index
     Handle ArithmeticException, ArrayIndexOutOfBoundsException, and InputMismatchException.

7. Write a Java program for a railway reservation system. The program should allow booking tickets for passengers. Handle exceptions when:
   - The passenger enters an invalid age
   - The requested number of seats exceeds available seats
   - The train number is invalid
     Use custom exceptions and demonstrate the use of throw and throws.

8. Write a Java program to demonstrate the use of throws keyword. Create methods for:
   - Reading a file
   - Parsing an integer from a string
   - Dividing two numbers
     Declare exceptions using throws and handle them in the main() method.

9. Write a Java program for employee salary processing. The program should calculate salary based on basic pay, bonus, and deductions. Handle exceptions for invalid salary values, null employee names, and invalid employee IDs. Use multiple catch blocks and a custom exception named InvalidEmployeeDataException.

10. Write a Java program to simulate a library management system. The system should allow issuing and returning books. Handle exceptions when:
    - A book is not available
    - The user enters an invalid book ID
    - The member ID is incorrect
    - The return date exceeds the due date

    Use custom exceptions, try-catch-finally, and proper exception messages.

Handling Multiple Exceptions

Java allows handling different types of exception using multiple cache blocks. Each catch block is used for a specific exception type.

Syntax:

```
try{
    //Code that may throw an exception
}
catch(IOException e){
```

```
      //Code to Handle IOException
    }
    catch(NumberFormatException e){
      //Code to handle NumberFormatException
    }
    catch(Exception e){
       //Handle any other exception
    }
```

Example of multiple catch block:

```
package ExceptionHandling;
import java.util.*;

/**
 * Write a description of class MultipleException here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class MultipleException
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String name = null;
        int num[] = {1,0,3};
        try{
            //System.out.print("\fnum[10]: "+num[10]);
            System.out.print("\fnum[2]: "+num[2]);
            System.out.print("\nEnter any value in num[2]: ");
            num[2] = sc.nextInt();
            name = "Joy";
            System.out.print("\nname =" + name.length());
            System.out.print("\nQuotient by dividing " + num[2] + "/" + num[1] +
            " = " + num[2]/num[1]);
        }
        catch(ArrayIndexOutOfBoundsException e){
            System.err.print("\nInvalid index value.\n ");
            //e.printStackTrace();
        }
        catch(InputMismatchException e){
            System.err.print("\nYou have not entered integer value.");
        }
        catch(NullPointerException e){
            System.err.print("\nYou have not entered any data in String variable");
        }
        catch(ArithmeticException e){
            System.err.print("\n Divide by zero error!");
        }
    }
}
```

The finally Block

The finally block is executed regardless of whether an exception occurs or not. It is commonly used to release resources like files or database connections.

Syntax:

```
try{
   //Code that may throw an exception
}
catch(Exception e){
   //Exception handling code
}
finally{
   //cleanup code
}
```

```
package ExceptionHandling;


/**
 * Write a description of class TryCatchFinally here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TryCatchFinally
{
    public static void main(String[] args){
        System.out.print("\f");
        try{
            int data = 25/0; //May throw ArithmeticException
            System.out.print("\ndata = " + data);
        }
        catch(ArithmeticException e){
            System.err.print("\nError: Division by zero not allowed.");
        }
        finally{
            System.out.print("\nI am the finally block, and I always executes");
        }
        System.out.print("\nProgram continues after finally block...");
    }
}
```

Output:

```
BlueJ: Terminal Window - AaditClass7
Options

I am the finally block, and I always executes
Program continues after finally block...

Can only enter input while your program is running

Error: Division by zero not allowed.
```

Revise:

Java try-catch block: The try block is used to enclose the code that might throw an exception during execution. Any code that has the potential to cause an error, such as division by zero, accessing an invalid array index, or reading a file, should be placed inside the try block.

The catch block is used to handle the exception thrown by the try block. You specify the type of exception to catch inside parenthesis. When an exception occurs in the try block, the corresponding catch block executes.

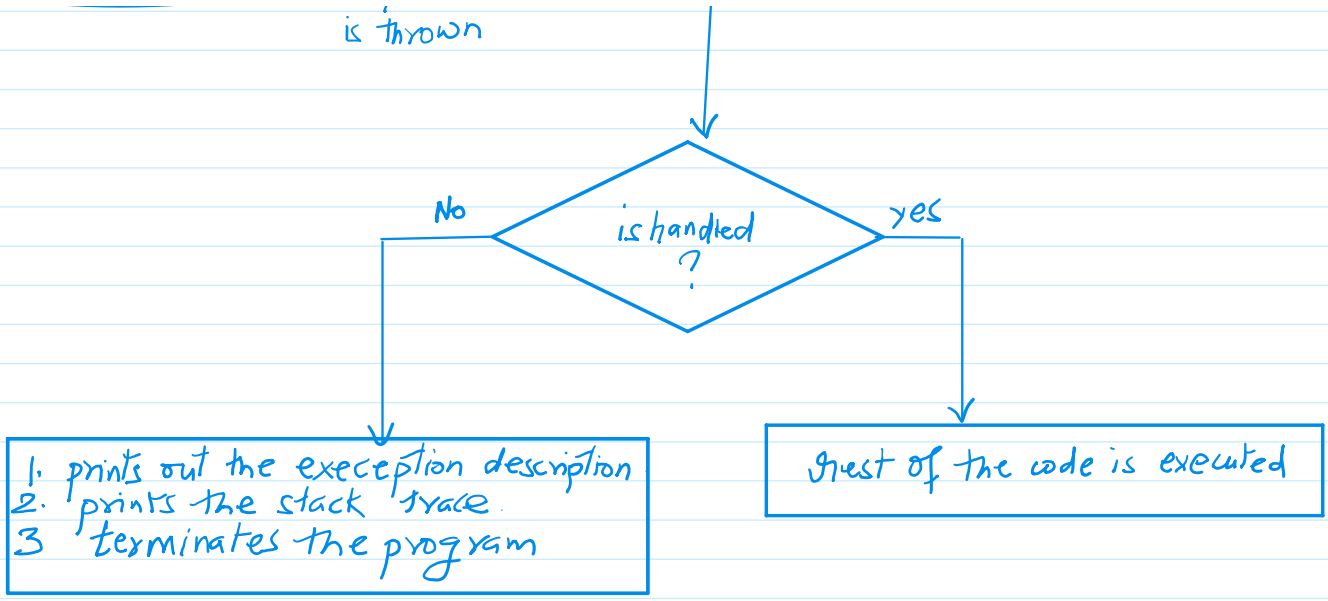
- The try block contains the code that might cause an exception.
- The catch block handles the exception if it occurs.
- Exception type can be a specific exception like ArithmeticException, IOException or the general Exception class.
- The catch block receives an exception object (commonly named E) That contains details about the exception.

How Java Handles Exceptions using try-catch block?
When an exception occurs, the Java virtual machine (JVM) First checks whether the exception is handled by the program.

- **If the exception is not handled:**
  - The JVM provides a default exception handler that performs the following task:
    - Prints the exception description.
    - Prints those stack trees (the hierarchy of methods where the exception occurred)
    - Terminates the program.
- **If the exception is handled by the programmer:**
    - The normal flow of the application is maintained and rest of the code continues to execute.

This mechanism makes sure that exception do not always crash the program and gives programmer a way to recover from runtime errors gracefully.

int data = 10/0; $\xrightarrow[\text{exception class is thrown}]{\text{an object of}}$ Exception object



Nested try-catch block

In Java, using a try block inside another try block is permitted. It is called a nested try block. Every statement that we enter a statement in try block context of that exception is pushed onto the stack.

For example, the inner try block can be used to handle ArrayIndexOutOfBoundException, while the outer try block can handle the ArithmeticException (division by zero).

Why use nested try Block?

Sometimes a part of a block may cause one type of exception, while the entire block itself may cause another. In such cases, nested try blocks are useful to handle multiple exceptions separately.

Syntax of Nested try-catch

A nested try block is a Tri block placed inside another Tri block. The outer try block can handle exception not caught by the inner try block(s).

```
try{
   //Outer try block
   //code that may throw an exception

   try{
      //inner try block
      //code that may throw an exception
   }
   catch([ExceptionType] e1){
      // Inner catch block handles ExceptionType.
   }
   // Optional more inner try-catch block can be added.
}
catch(ExceptionType2 e){
   // Powder catch block handles exception not caught by inner try block.
}
```

* Outer try block contains the main code that may throw exceptions.
* If an exception occurs that is not handled by any inner try-catch, it is passed to this outer block.
→ The inner try block(s) is/are placed inside the outer try-block.
* These can handle exception specific to the inner code.
* Each try block (inner or outer) must have at least one corresponding catch block to handle exceptions.
* The outer catch block act as a backup handler for any

For example:

```
import java.util.*;
```

* Each try block (i.e. outer try) must have at least one corresponding catch block to handle exceptions.

For example:

* The outer catch block act as a backup handler for any exception not handled by inner catch blocks.

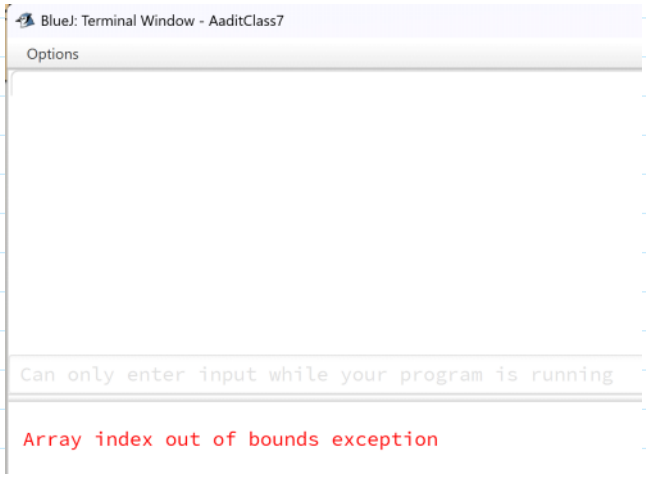
```
import java.util.*;

public class ArrayAccess{
public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        System.out.print("\f");
        double arr[] = new double[6];
        for(int i=0;i<6;i++){
            try{
                System.out.print("\nEnter the Numerator : ");
                int n = sc.nextInt();
                System.out.print("\nEnter the Denominator : ");
                int d = sc.nextInt();
                try{ //nested try
                    int r = n/d;
                    arr[i] = r;
                }
                catch(ArithmeticException ae){ //nested catch
                System.err.print("\nDividing by 0 is incorrect ");
            }
            }
            catch(InputMismatchException ime){ //outer-catch
                System.err.print("\nInput should be in numerical format ");
                i--;
                sc.nextLine();
            }
        }
        System.out.print("\n\n[");
        for(int j=0;j<6;j++){
            System.out.print(arr[j] + " ");
        }
        System.out.print("]");
        try{
            System.out.print("\nEnter Index Of Array Element(1-6) : ");
            sc.nextLine();
            int ind = sc.nextInt();
            System.out.print("\nArray Element : " + arr[ind-1]);
        }
        catch(ArrayIndexOutOfBoundsException aie){
            System.err.print("\nIncorrect Index has been entered ");
        }
    }
```

Another example:

```
package ExceptionHandling;
import java.util.InputMismatchException;


/**
 * Write a description of class NestedTryBlockOne here.
 *
 * @author (your name)
```

```
 * @version (a version number or a date)
 */
public class NestedTryBlockOne
{
    public static void main(String[] args){
        try{
            try{
                try{
                    int arr[] = {1, 2, 3, 4};
                    System.out.print("\n"+arr[10]);
                }
                catch(ArithmeticException e){
                    System.err.print("\nArithmetic exception");
                }
            }
            catch(InputMismatchException e){
                System.err.print("\nInput mismatch exception");
            }
        }
        catch(ArrayIndexOutOfBoundsException e){
            System.err.print("\n Array index out of bounds exception");
        }
        catch(Exception e){
            System.err.print("\nException");
            System.err.print("\nHandled the main try-block");
        }
    }
}
```

Output:



finally Block in Java

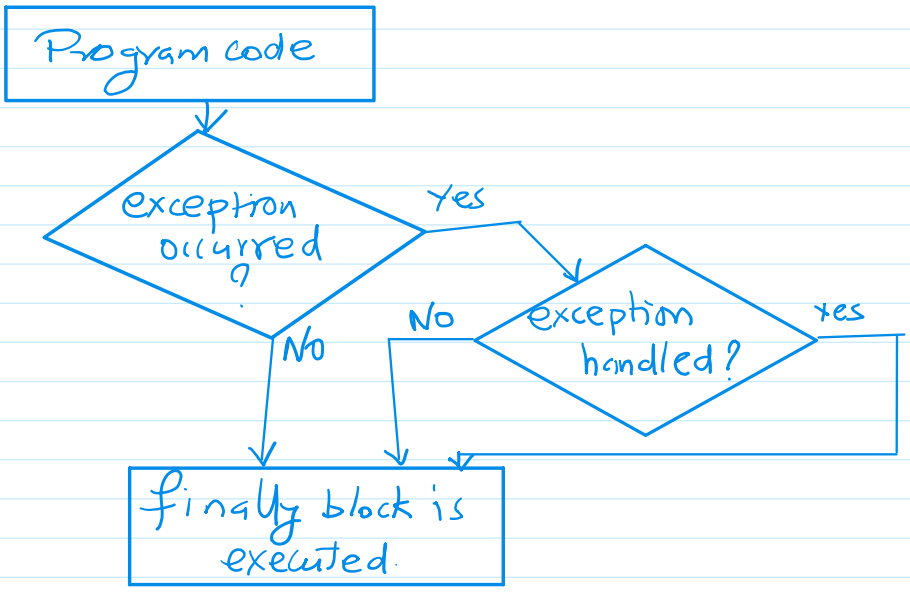
In Java, the finally block is used to execute important codes such as closing files or releasing resources regardless of whether an exception occurs or not.

What is the finally Block in Java?
The finally block is a block of code that always executes after a try block, whether an exception thrown or not. The finally block is useful when you want to clean up

activities or free the resource, like closing files, streams or database connections.

Key points

1. A **finally block** is optional, but when present it always executes after the try and catch blocks.
2. It executes even if a return statement is encountered in the try or catch blocks.



For example:

```
package ExceptionHandling;


/**
 * Write a description of class FinallyBlockone here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class FinallyBlockOne
{
    public static void main(String[] args){
        try{
            int data = 100/0;
        }catch(ArithmeticException e){
            System.err.print("\nDivide by 0 is not valid in maths");
        }finally{
            System.out.print("\nFinally Block Executed: Clean up code runs
here.");
        }
        System.out.print("\nRest if the program continues...");
    }
}
```

Output:

```
BlueJ: Terminal Window - AaditClass7
Options

Finally Block Executed: Clean up code runs here.
Rest if the program continues...

Can only enter input while your program is running

Divide by 0 is not valid in maths
```

Another example, where catch block is not executed:

```
package ExceptionHandling;


/**
 * Write a description of class FinallyBlockone here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class FinallyBlockOne
{
    public static void main(String[] args){
        System.out.print("\f");
        try{
            int data = 100/7;
            System.out.print("\nQuotient: " + data);
        }catch(ArithmeticException e){
            System.err.print("\nDivide by 0 is not valid in maths");
        }finally{
            System.out.print("\nFinally Block Executed: Clean up code runs here.");
        }
        System.out.print("\nRest if the program continues...");
    }
}
```

Output:

```
BlueJ: Terminal Window - AaditClass7
Options

Quotient: 14
Finally Block Executed: Clean up code runs here.
Rest if the program continues...
```